COMPUTER SCIENCE TRIPOS Part IB – 2022 – Paper 5

6 Introduction to Computer Architecture (swm11)

SystemVerilog and FPGAs

The Thrupenny Bit FPGA company produces FPGAs with a sea of logic elements (LEs) depicted below (Fig 1) containing 3-bit LUTs (LookUp Tables) and a D flip-flop (DFF). Each LUT can be programmed with any Boolean function of three inputs and one output. The output of the LUT can be sent over the programmable wiring or to the input of the DFF. The DFF has data (D), asynchronous level-sensitive clear and edge triggered clock inputs, and the Q output. The clear input can be used to reset the DFF to zero.

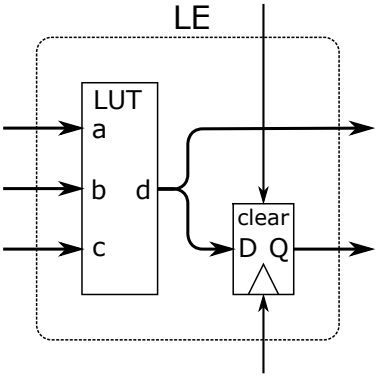


Fig. 1: Thrupenny Bit LE

```
module counter(input  logic       clk,
               input  logic       rst,
               input  logic       enable,
               output logic [2:0] count);

  always_ff @(posedge clk or posedge rst)
    if(rst)
      count <= 0;
    else
      if(enable)
        count <= count+3'd1;
endmodule
```

Fig. 2: SystemVerilog code

(*a*) What is the minimum number of LEs required to implement the `counter` module in Fig. 2 on a Thruppenny Bit FPGA? Provide a detailed rationale. [8 marks]

Answer: The `counter` module has three state bits (`count[2:0]`) and we need a DFF for each bit, so we need at least three LEs to implement the module. There are many implementations involving more than three LEs, but is it possible to implement the design with just three LEs?

Yes. First we observe that we can use the `clear` inputs on the DFFs to implement reset, i.e. we connect the `rst` signal to every `clear` input for each LE.

The next state of `count[0]` (`count[0]'`) requires only `enable` and the current state `count[0]`:
`count[0]' = enable ? !count[0] : count[0]`

For `count[1]'` we need to know the current value of `count[1]` and whether `count[0]` will change from 1 to 0, i.e. `count[0]` will overflow:
`count[1]' = count[0] && !count[0]' ? count[1]+1`

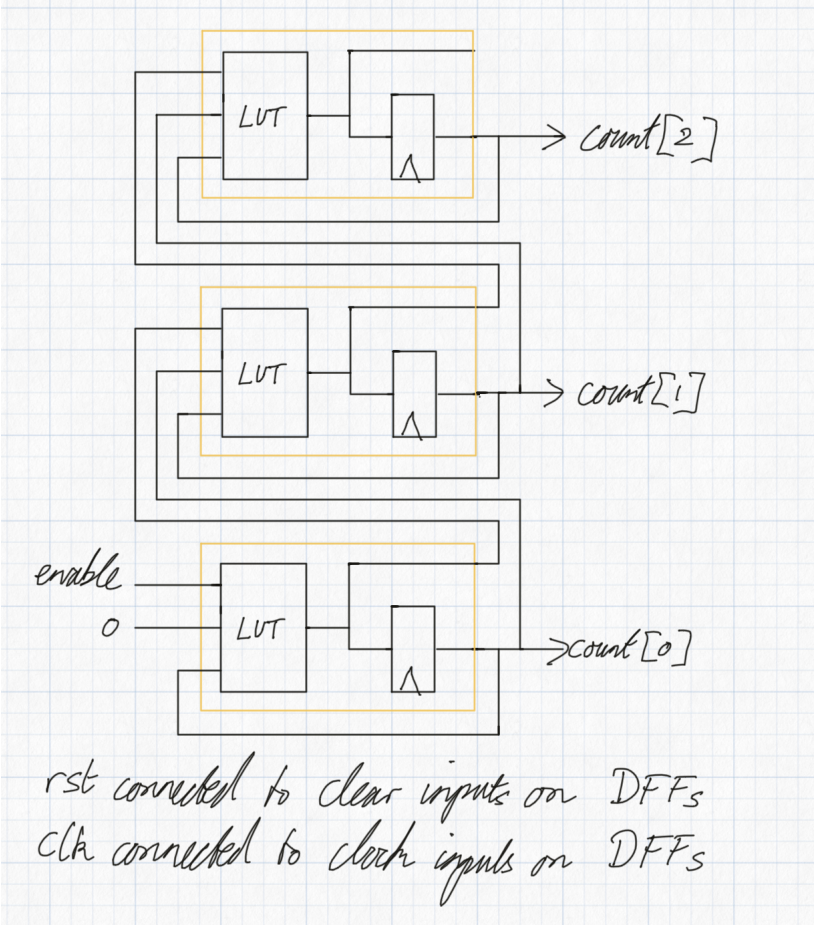
Note that we do not need the `enable` bit as part of this state machine since `count[0]` will only change when enabled.

`count[2]` requires the same logic as `count[1]`:
`count[2]' = count[1] && !count[1]' ? count[2]+1`

So yes, it can be done using 3, as shown in the next part.

(*b*) Produce a circuit diagram of your implementation showing how the LEs are wired together. [6 marks]

Answer:



(*c*) Tabulate the contents of the LUTs for each LE used (i.e. provide a state transition table for each LUT).

Answer:

LUT0

unused	enable	count[0]	count[0]'
x	0	0	0
x	0	1	1
x	1	0	1
x	1	1	0

LUT1

count[0]	count[0]'	count[1]	count[1]'
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

LUT2

count[1]	count[1]'	count[2]	count[2]'
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

[6 marks]